

Module 04 Assignment

Introduction to Programming Language with AI (JavaScript + Node.js)

Name	Shadot Hossan Riyad
Phone	01706877681
Batch	21 st

Q1. JavaScript Coding Practice

1. Check Whether a Number is Even or Odd

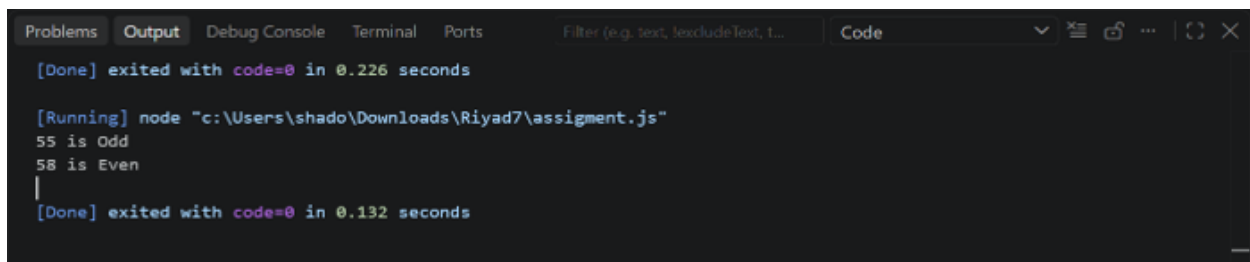
Problem Statement:

Write a JavaScript program that takes a number as input and determines whether it is even or odd.

Source Code:

```
function checkNum(num){
    if(num % 2 === 0)
    {
        return `${num} is Even`;
    } else{
        return `${num} is Odd`;
    }
}
console.log(checkNum(55));
console.log(checkNum(58));
```

Output:



Explanation:

The program uses the modulus operator (%) to find the remainder when the number is divided by 2. If the remainder is 0, the number is even; otherwise, it is odd. This is the simplest and most efficient way to check even/odd in JavaScript, and the same modulus logic is often reused in automation scripts (e.g., validating every alternate row/record in a table).

2. Find the Largest of Three Numbers

Problem Statement:

Write a JavaScript program that accepts three numbers and returns the largest among them.

Source Code:

```
function findBiggestNum(a, b, c){
    if (a >= b && a >= c){
        return `${a} is the Biggest Number.`;
    } else if (b >= a && b >= c){
        return `${b} is the Biggest Number.`;
    } else {
        return `${c} is the Biggest Number.`;
    }
}
console.log(findBiggestNum(4,5,9));
console.log(findBiggestNum(10,4,9));
console.log(findBiggestNum(11,55,19));
```

Output:



Explanation:

The function compares each number against the other two using logical AND (&&) conditions. Using >= (instead of >) correctly handles the case where two numbers are equal (e.g., 5, 5, 3), which is an important edge case to consider when writing test-worthy code.

3. Reverse a String

Problem Statement:

Write a JavaScript program that reverses a given string.

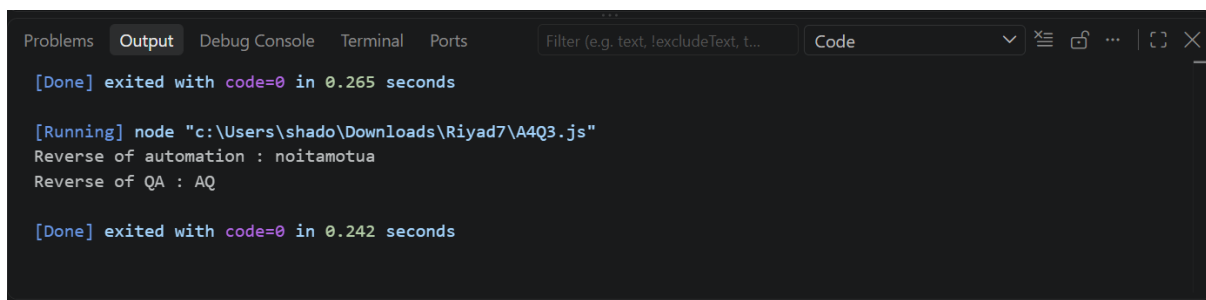
Source Code:

```
function reverseString(text1) {
    return text1.split('').reverse().join('');
}
let text1 = "automation";

console.log(`Reverse of ${text1} : ${reverseString(text1)}`);

text1 = `QA`;
console.log(`Reverse of ${text1} : ${reverseString(text1)}`);
```

Output:



```
[Done] exited with code=0 in 0.265 seconds

[Running] node "c:\Users\shado\Downloads\Riyad7\A4Q3.js"
Reverse of automation : noitamotua
Reverse of QA : AQ

[Done] exited with code=0 in 0.242 seconds
```

Explanation:

JavaScript strings do not have a built-in reverse() method, so the string is first split into an array of characters using split(""), reversed using reverse(), and then joined back into a string using join(""). This split -> transform -> join pattern is very common in JavaScript array/string manipulation.

4. Count Vowels in a String

Problem Statement:

Write a JavaScript program that counts the number of vowels (a, e, i, o, u) present in a given string.

Source Code:

```
function countVowels(str) {
  const vowels = 'aeiouAEIOU';
  let count = 0;
  for (let char of str) {
    if (vowels.includes(char)) {
      count++;
    }
  }
  return count;
}

let str = "Riyaad"
console.log(`Vowels in ${str}: ${countVowels(str)}`);

str = "Shadot Hossan Riyad";
console.log(`Vowels in ${str}: ${countVowels(str)}`);
```

Output:



```
[Done] exited with code=0 in 0.242 seconds

[Running] node "c:\Users\shado\Downloads\Riyad7\Q4.js"
Vowels in Riyaad: 3
Vowels in Shadot Hossan Riyad: 6

[Done] exited with code=0 in 0.183 seconds
```

Explanation:

The program loops through each character of the string using a for...of loop and checks whether the character exists in the vowels string using includes(). Both uppercase and lowercase vowels are included in the check to avoid the common bug of under-counting due to case sensitivity.

5. Remove Duplicate Values from an Array

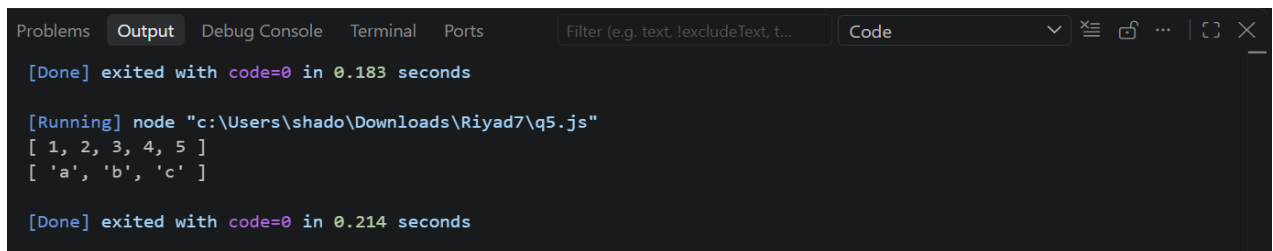
Problem Statement:

Write a JavaScript program that removes duplicate values from an array.

Source Code:

```
function removeDuplicates(arr) {  
    return [...new Set(arr)];  
}  
  
console.log(removeDuplicates([1, 2, 2, 3, 4, 4, 5]));  
console.log(removeDuplicates(['a', 'b', 'a', 'c', 'b']));
```

Output:



```
Problems Output Debug Console Terminal Ports Filter (e.g. text, !excludeText, t...) Code  
[Done] exited with code=0 in 0.183 seconds  
  
[Running] node "c:\Users\shado\Downloads\Riyad7\q5.js"  
[ 1, 2, 3, 4, 5 ]  
[ 'a', 'b', 'c' ]  
  
[Done] exited with code=0 in 0.214 seconds
```

Explanation:

A JavaScript Set object can only store unique values, so passing the array into new Set() automatically removes duplicates. The spread operator (...) is then used to convert the Set back into a regular array. This is a modern and concise alternative to the older nested-loop or filter()+indexOf() approaches.

Q2. Introduction to Node.js

1. What is Node.js?

Node.js is a runtime environment that allows JavaScript code to run outside a web browser, such as on a server, a local machine, or inside a terminal. It is built on Google Chrome's V8 JavaScript engine, which compiles JavaScript directly into fast machine code. Before Node.js existed, JavaScript could only run inside a browser; Node.js made it possible to use JavaScript as a general-purpose, server-side programming language.

2. How Does Node.js Work?

Node.js works on a single-threaded, event-driven, non-blocking I/O model. Instead of waiting for a slow operation (such as reading a file, querying a database, or calling an API) to finish before moving on, Node.js hands the operation off to the system and continues executing other code. Once the operation completes, a callback function (or a Promise/async-await) is triggered to handle the result. This mechanism is managed by the Event Loop, which allows Node.js to efficiently handle many operations at the same time without creating a new thread for each one.

3. Advantages of Node.js for QA Automation

1. Playwright, Cypress, and most modern JavaScript-based test automation frameworks are built on top of Node.js, so Node.js is required to run them.
2. Its non-blocking, asynchronous model is well suited to automation, where multiple browser actions or API requests can be handled concurrently using `async/await`.
3. npm (Node Package Manager) is bundled with Node.js and is used to install almost the entire QA automation toolchain: Playwright, Mocha, Chai, Supertest, ESLint, reporting libraries, and more.
4. It allows QA engineers to write test scripts, utility scripts, and even simple API mock servers using a single language: JavaScript.
5. It has a very large open-source ecosystem, meaning most automation problems (reporting, parallel execution, CI integration) already have a ready-made npm package.

4. Difference Between Browser JavaScript and Node.js

Aspect	Browser JavaScript	Node.js
Environment	Runs inside a web browser (Chrome, Firefox, etc.)	Runs on a server or local machine, outside a browser
Access to DOM	Yes - can access documents and window objects	No DOM access - no document or window object
File System Access	Not allowed (sandboxed for security)	Allowed, using the built-in fs module
Module System	ES Modules (import / export), browser-dependent	CommonJS (require / module.exports) traditionally; ES Modules also supported
Networking	Limited (fetch/XHR, same-origin restrictions apply)	Full control over networking (http, net modules)
Typical Use in QA	Rarely used directly; occasionally injected via <code>page.evaluate()</code> in Playwright	Core environment - the entire Playwright/API test suite runs on Node.js